START HERE • 10-MINUTE REVIEW

# Your project in one page

Walmart interview study guide | Machine Translation Lab | Prepared October 13, 2026. Based on all four current notebooks, their saved outputs, the saved training state, environment checks, and your supplied job description.

Core message: I built and examined a small English-to-French translation workflow. The valuable result was learning how data quality, training objectives, decoding, and business checks can disagree. The experiment demonstrates the workflow; it does not establish production readiness.

| Remember | Evidence |
|---|---|
| Task and model | English to French; google-t5/t5-small; a pretrained encoder-decoder Transformer. |
| Data | OPUS Books, en-fr: 127,085 sentence pairs. Book prose differs from retail language. |
| Tiny training run | 128 training examples; 32 validation examples; 1 epoch; batch size 1; CPU; 128 optimizer steps recorded. |
| Validation result | Loss: 2.093639 -> 2.065281. BLEU: 16.5116 -> 15.8853. Lower loss did not mean better generated translations. |
| Data issue | Initial 80/20 split: 461 repeated pairs in train, 64 in test, and 119 distinct pairs shared across both. |
| Retail exercise | 6/6 automatic checks passed; 0 human-reviewed; BLEU 72.56; mean 0.114 s, maximum 0.146 s. Decision: HOLD. |
| Checkpoint distinction | Notebook 3 trained and saved a model. Notebook 4 still uses pretrained T5-small. |

## Read in this order

Pages 2-3: interview story and project map. Pages 4-8: data, tokenization, model, training, and metrics. Pages 9-13: actual results, errors, inference, retail checks, and debugging. Pages 14-16: proposed improvements, scale, and Haitian Creole. Pages 17-18: practice questions, study schedule, and sources.

Evidence rule: numbers here are saved project results, not a new training run. Proposed extensions are labeled. Some setup prose is older than the executed notebooks. Use current code and saved artifacts to explain what happened.

# Tell the project story

## A 60-90 second answer to practice

"I worked through an English-to-French translation lab using T5-small and OPUS Books. I inspected the data, tokenized paired sentences, ran a small CPU fine-tuning experiment, and compared generated translations with SacreBLEU and targeted retail checks. The run used 128 training examples and 32 validation examples. Validation loss fell from about 2.094 to 2.065, but BLEU fell from 16.51 to 15.89, so I would not call it a quality improvement. I also found duplicate translation pairs across the initial split and a language-direction problem in an error-analysis example. For retail, I checked quantities, units, brands, price formatting, and negation. My next step would be a deduplicated, language-verified retail evaluation set with bilingual review, followed by controlled quality and latency comparisons."

Adapt this script to work you personally completed and can explain. The repository cannot establish who authored each cell or any experience outside this lab.

| The supplied role asks for | How to connect this lab |
|---|---|
| Translation and localization systems | Explain the entire pipeline, error categories, and preservation of product and policy meaning. |
| Applied NLP research | Frame a hypothesis, control a baseline, inspect failures, and report a negative result honestly. |
| Real-time and batch data pipelines | Use the scaling design on page 15 as a proposed extension; no service is deployed here. |
| Product and business partnership | Turn a use case into measurable quality, latency, cost, and review requirements. |
| Seq2seq models or LLM applications | Explain T5 and compare approaches conceptually; this lab does not benchmark an LLM. |
| English and Haitian Creole fluency | The lab is English-French. State your actual language abilities; no Haitian Creole evaluation is present. |

## A useful answer structure

For each question, answer: decision -> reason -> observed evidence -> limitation -> next experiment. Example: "I used a small CPU baseline to validate the workflow. It completed 128 steps, but its validation BLEU declined, so I would first improve evaluation data before increasing training."

Role source: ML_Scientist_JD_Translation.pdf, one-page supplied description. Its company claims are background from that document, not independently verified current facts.

# Map the entire project

| File or directory | What to know |
|---|---|
| notebooks/01_dataset_exploration.ipynb | Loads OPUS Books, creates an 80/20 split, inspects pairs, checks empty text, duplicates, and overlap. |
| notebooks/02_tokenization.ipynb | Loads T5 tokenizer, prefixes inputs, creates labels, dynamically pads a four-example batch, and checks tensor shapes. |
| notebooks/03_model_evaluation.ipynb | Defines BLEU callback, creates train/validation/test splits, fine-tunes, saves, performs error analysis and six retail checks. |
| notebooks/04_model_inference.ipynb | Loads pretrained T5-small, samples outputs, compares greedy and four-beam decoding, checks generation invariants. |
| outputs/t5-small-en-fr/ | checkpoint-128 contains weights and training-resume state. final contains exported model and tokenizer. |
| tests/test_environment.py | Offline environment, API, synthetic T5, metric, notebook schema, and kernel checks. These are not accuracy tests. |
| README.md; requirements.txt; .vscode/ | Setup guidance, pinned dependencies, interpreter/kernel configuration. Some README scope claims are stale. |
| src/; data/; other outputs/ | src has an initializer, not a production pipeline. data includes caches. Other outputs contain setup records, backups, and validation aids. |

## The training flow

```
paired English/French text
  -> clean and split
  -> add task prefix to English
  -> tokenize source and target
  -> collate padded tensors
  -> model forward pass -> loss -> backpropagation
  -> generate on validation -> decode -> BLEU + review
  -> save model and tokenizer
```

## The inference flow

New English text -> same prefix and tokenizer -> model.generate -> decoded French -> automatic checks and human review. No reference translation is needed to generate. References are needed for reference-based evaluation.

Each notebook has its own kernel state. Notebook 3 recreates its dataset and tokenization rather than relying on variables from notebooks 1 or 2. Notebook 4 does not automatically discover the exported fine-tuned checkpoint.

NOTEBOOK 1

# Dataset exploration and leakage

A parallel corpus contains aligned source and target text. Here each row has an id and a translation dictionary with en and fr fields. The saved dataset contains 127,085 rows and initially exposes a train split only.

| Initial split check | Saved result | Interpretation |
|---|---|---|
| Train / test | 101,668 / 25,417 | 80/20 split with seed 42. |
| Empty pairs | 0 / 0 | Checks stripped strings; does not prove alignment or language correctness. |
| Repeated pairs within split | 461 / 64 | Count = number of rows minus number of unique stripped pairs. |
| Shared pairs across splits | 119 | Distinct identical English/French pairs occur in both sets. This is text leakage. |

## Why this matters

Leakage gives an evaluation model information too similar to its training examples and can make generalization look better than it is. Different row IDs do not guarantee different content. The later notebook verifies ID separation, but it does not deduplicate the text first. The 119 count belongs to the full initial 80/20 split; overlap for the actual 128/32 experiment is not reported.

## The later 80/10/10 split

Notebook 3 splits the original data 80/20, then divides the holdout in half, both with seed 42. This implies 101,668 train, 12,708 validation, and 12,709 test rows. Only the first 128 rows of the shuffled training split and 32 validation rows are selected. Test is reserved; no final test score is shown.

## What you would improve - proposed

- Normalize conservatively and group exact duplicate source/target pairs before splitting. Check repeated sources with differing targets as well.
- Where metadata permits, split by book/document or related product family to reduce near-duplicate and contextual leakage.
- Audit language direction, pair alignment, length ratios, long sequences, markup, and representative samples. Flag cases for review instead of assuming every outlier is wrong.
- Build a separate retail-domain test set. Book prose alone cannot establish performance on product titles, attributes, returns, or warnings.

Interview answer: "A fixed seed makes the split repeatable. It does not make the evaluation valid. I found exact text overlap, so I would fix content-level leakage and language alignment before treating small score changes as meaningful."

NOTEBOOK 2

# Tokens, masks, labels, and padding

The tokenizer converts strings to subword IDs; it does not translate by itself. T5 uses a SentencePiece-based vocabulary. Subwords help represent unfamiliar words, but a valid tokenization does not guarantee correct meaning.

```
prefix = "translate English to French: "
model_inputs = tokenizer(
    [prefix + pair["en"] for pair in pairs],
    text_target=[pair["fr"] for pair in pairs],
    max_length=128, truncation=True
)
```

| Field / setting | Meaning |
|---|---|
| input_ids | Token IDs for prefixed English. The prefix tells the multitask model which task to perform. |
| attention_mask | 1 for real input tokens and 0 for padding in this lab; keeps padded positions from being treated as source content. |
| labels | French token IDs used as training targets. They are not inference inputs. |
| pad_token_id / EOS | T5 padding is ID 0; EOS is ID 1. EOS marks sequence end. -100 is a loss-ignore value, not a vocabulary token. |
| Dynamic padding | DataCollatorForSeq2Seq pads each batch to its longest source and longest target, separately. Label padding defaults to -100. |
| Truncation | Both source and target are limited to 128 tokens in preprocessing. This can remove important information. |

## Actual batch evidence

For four examples, input_ids and attention_mask have shape [4, 13]; labels have shape [4, 8]. Source and target lengths can differ. The decoded first source was "translate English to French: The Wanderer" and the target was "Le grand Meaulnes". A book-title translation may not be word-for-word.

## Important distinctions

- Padding adds placeholders; truncation removes tokens. Padding is usually computational overhead; truncation can change meaning.
- An attention mask controls attention to padded inputs. A -100 label tells the loss function not to score that target position. Neither replaces the other.
- For decoding labels, filter out -100 or replace it with pad_token_id. In notebook 3, also sanitize generated arrays because Trainer can add -100 while combining batches.
- Pass a loaded model to the collator, or omit model. Passing a checkpoint-name string is not the same as passing a model object.

Map is batched preprocessing, not gradient training. Notebook 2 retains raw dataset columns; its collation cell deliberately selects only input_ids, attention_mask, and labels. Notebook 3 removes raw columns during mapping.

# How T5 learns a translation

T5 frames language tasks as text in and text out. Its encoder reads the input sequence; its decoder generates an output sequence. A task prefix selects English-to-French translation. Fine-tuning starts from pretrained weights rather than learning every language pattern from scratch. [R1]

## Attention, in practical terms

- Encoder self-attention lets source tokens use surrounding source context. This matters when a word has multiple meanings.
- Decoder causal self-attention uses the output prefix without looking at future output tokens.
- Cross-attention lets each decoder step consult the encoded source. It helps connect the French output to English content.

```
P(y | x) = product over t of P(y_t | y_<t, x)
Loss = - mean log P(correct target token | context)
       over target positions not labeled -100
```

Attention uses learned query, key, and value representations to weight relevant information. The probability factorization explains why output is generated token by token. The loss rewards higher probability on the correct reference token.

## Teacher forcing versus generation

During supervised training, the decoder is given the correct earlier target tokens, shifted right. For T5, the decoder start token is the pad token. During inference, the decoder instead conditions on its own prior predictions. An early error can affect later tokens. This difference helps explain why lower validation loss need not improve the final generated sentence.

## Training versus inference

| Training | Inference |
|---|---|
| Computes loss against labels. | Generates without knowing the target. |
| Backpropagation produces gradients; optimizer changes weights. | Weights stay fixed. |
| Training mode may enable dropout. | model.eval() changes training-dependent behavior. |
| Needs gradient tracking. | torch.inference_mode() disables gradient tracking and related overhead. |

model.eval() and torch.inference_mode() solve different problems; use both for the inference cells. The project tests a tiny randomly initialized T5 to verify the backend, but those synthetic outputs cannot demonstrate translation quality.

> Explain simply: "The encoder builds a contextual representation of English. The decoder predicts French one token at a time, attending to that representation. Fine-tuning adjusts probabilities using paired examples; decoding decides which output sequence to choose."

# Fine-tuning configuration

The executed training cell evaluates a pretrained baseline, calls trainer.train(), evaluates again, then saves the model and tokenizer. Older markdown saying training remains commented out is stale. The actual call is active and a checkpoint at step 128 exists.

| Setting | Project value | Why it matters |
|---|---|---|
| Training / validation examples | 128 / 32 | Small CPU workflow experiment; limited statistical evidence. |
| Epochs / training batch | 1 / 1 | One pass through 128 training examples; saved global_step is 128. |
| Learning rate | 2e-5 | Controls optimizer update size; no search over this value is shown. |
| Weight decay | 0.01 | Regularizes parameter updates; does not by itself prevent overfitting. |
| Device and precision | use_cpu=True; fp16=False; bf16=False | No GPU acceleration or mixed-precision speedup is demonstrated. |
| Evaluate / save | At each epoch; save_total_limit=1 | Records an epoch-end checkpoint; this is not best-BLEU model selection. |
| Generated metrics | predict_with_generate=True | BLEU is computed from decoded generation, not raw logits. |
| Generation limit | generation_max_length=128 | Limits decoder sequence length during Trainer evaluation. |
| Reproducibility / reporting | seed=42; report_to=none; push_to_hub=False | Local experiment; no model upload or external tracking shown. |

### Read the training sequence

```
baseline_metrics = trainer.evaluate()
train_result = trainer.train()
trained_metrics = trainer.evaluate()
trainer.save_model(str(checkpoint_dir))
tokenizer.save_pretrained(str(checkpoint_dir))
```

The same validation subset is used before and after training. Reuse is appropriate for model development; the separately reserved test set should remain untouched until a final choice is frozen.

### What the saved folders mean

checkpoint-128 contains weights, tokenizer/config files, optimizer state, scheduler state, RNG state, and trainer_state.json. These support resuming training. final contains exported model and tokenizer assets; it is the intended folder for loading the fine-tuned model for inference. Notebook 4 currently points elsewhere.

> Interview answer: "I deliberately started with a tiny run to verify the workflow. I would not infer a training-data scaling law, optimal hyperparameters, or deployment quality from one epoch on 128 examples."

# BLEU and the evaluation callback

BLEU measures modified n-gram precision against reference text, with a brevity penalty for overly short output. It usually combines orders 1 through 4. SacreBLEU standardizes scoring and exposes a signature for reproducibility. A BLEU score is not the percentage of correctly translated sentences. [R2, R3]

```
BLEU = 100 * BP * exp(sum_n w_n * log(p_n))
BP = 1 if candidate length > reference length
     exp(1 - reference_length/candidate_length) otherwise
```

p_n is clipped n-gram precision: repeating a word cannot earn unlimited credit. Smoothing and short-sentence handling affect edge cases. Corpus BLEU aggregates counts across a corpus; it is not the average of sentence BLEU scores.

## Your callback, step by step

- Receive predictions and labels; unwrap predictions if Trainer supplied a tuple.
- Replace -100 with tokenizer.pad_token_id in both arrays before batch_decode.
- Decode with skip_special_tokens=True and strip surrounding whitespace.
- Call metric.corpus_score(decoded_preds, [decoded_labels]). The outer list represents one reference stream.
- Return BLEU and average generated non-pad token count. gen_len counts tokens, potentially including EOS; it is not a word count.

| API | One-reference format for two examples |
|---|---|
| Direct SacreBLEU, used here | predictions = [p1, p2]; references = [[r1, r2]] |
| Hugging Face Evaluate wrapper | predictions = [p1, p2]; references = [[r1], [r2]] |

## What BLEU misses

Alternative valid wording may score poorly. A fluent sentence can score reasonably while losing a brand or reversing a condition. Misaligned references can punish correct behavior. The small sentence-level diagnostic uses effective_order=True, but sentence scores are only prompts for inspection.

## A stronger evaluation bundle - proposed

Use corpus BLEU plus chrF (character overlap), a suitable learned metric such as COMET, and bilingual human error review. COMET models estimate translation quality using learned representations; select a model with appropriate language coverage and validate it for the domain. Neither chrF nor COMET is implemented in this lab. [R2, R4]

Record dataset and split identity, checkpoint, decoding settings, metric version/signature, and reference preparation. Here references are decoded tokenized labels; target truncation may alter them. Audit truncation and score against full reviewed references for a final benchmark.

# Interpret the training result

| Saved validation measure | Before | After | Change |
|---|---|---|---|
| Loss | 2.093639 | 2.065281 | -0.028358 |
| Corpus BLEU | 16.5116 | 15.8853 | -0.6263 points |
| Mean generated token count | 52.2188 | 52.3125 | +0.0937 |

The post-training loss and BLEU are also recorded in checkpoint-128/trainer_state.json. That file records epoch 1.0 and global_step 128. The baseline numbers come from the saved before/after table in notebook 3.

## What the result supports

The model completed a small supervised update and the evaluation pipeline produced metrics. On the 32-example validation subset, reference-token loss decreased slightly and generated-text BLEU decreased. The correct conclusion is that this run did not demonstrate improved generated translation quality.

## Why loss and BLEU can move differently

- Loss evaluates probabilities of reference tokens with the correct prior target context. BLEU evaluates discrete generated sequences with the model's own prior outputs.
- A small probability change can change the chosen token and all later tokens. Conversely, probability can improve without changing the chosen sentence.
- BLEU is sensitive to overlap, references, and length. Its objective differs from token cross-entropy.
- Thirty-two examples, noisy alignment, duplicate content, and truncation make a small change hard to interpret.

## What the result does not establish

- No statistically reliable degradation or improvement is demonstrated; no confidence interval or significance analysis is saved.
- Overfitting is a possible hypothesis, not a proven diagnosis. There is no informative multi-epoch train/validation curve here.
- The six-example retail BLEU of 72.56 is not comparable to the book-validation BLEU of 15.8853. They use different content, sample sizes, and evaluation paths.
- No held-out final test quality, live customer impact, revenue uplift, or production latency claim is supported.

## What to say if challenged

> "I expected a workflow check, not a strong quality gain. The loss/BLEU disagreement made me inspect the evaluation rather than choose the flattering number. Before more training, I would fix language alignment and leakage, collect a representative validation set, and compare checkpoints under identical generation settings."

Potential follow-up: use paired resampling over the same examples to quantify score uncertainty once the evaluation set is credible. A statistical procedure cannot repair leaked or mislabeled data.

# Read errors before blaming the model

The error-analysis cell generates on ten validation examples and displays the five lowest sentence BLEU scores. It uses the current trainer model after the saved training run. Inspect source, reference, and prediction together.

| Observed example | Diagnosis / lesson |
|---|---|
| ID 76401: en field says "Ce jour-là il avait horreur de parler."; fr reference says "That day he had a horror of speaking." Prediction repeats the French. | The displayed language direction is reversed. It scores 4.87 and is flagged copied source, but this is first an evaluation-data problem. One bad row does not prove the whole corpus is reversed. |
| "GREAT SURPRISE ON THE PART OF THE AUDIENCE." -> "PRINCIPAUX SUR LA PARTIE DE LA VÉRIFICATION." | Major meaning error; sentence BLEU 5.52. Automatic heuristic flags show none detected. Nonempty text is not adequate translation. |
| "That's another pair of shoes!" -> "C'est une autre paire de chaussures!" | Likely literal rendering of an idiom; the supplied reference is "C'est autre chose!". Context and idiomatic meaning require review. |
| "We had better not mention it." -> "Nous ne l'avons pas mentionné." | The English recommendation is rendered as a past-tense statement. Negation alone can survive while tense and intent change. |

## The diagnostic heuristics

The code flags empty output, changed digit tokens, exact source copying, unusually short/long output relative to the reference, and source truncation. These prioritize review. They are neither complete detectors nor confirmed errors.

## A practical annotation scheme - proposed

- Adequacy: is any meaning omitted, added, or contradicted? Mark the affected span.
- Fluency: grammar, agreement, spelling, and natural phrasing.
- Retail fidelity: brand/SKU, number, unit, price, attribute, negation, policy condition.
- Reference/data defect: wrong language, wrong alignment, incomplete or questionable reference. Keep this separate from model error.
- Severity: distinguish minor style differences from errors that could change a purchase decision or instruction. Agree the rubric with reviewers.

Strong interview point: "A low metric score is a signal to investigate. I found both an apparent mislabeled language pair and genuine meaning errors. I would correct the evaluation data and report model errors by category and severity."

# Inference and decoding tradeoffs

Notebook 4 loads google-t5/t5-small on CPU and calls model.eval(). Generation runs inside torch.inference_mode(). It adds the translation prefix, tokenizes with padding and a 128-token input limit, generates, then decodes with skip_special_tokens=True.

| Strategy | Settings here | Tradeoff |
|---|---|---|
| Sampling example | do_sample=True; top_k=30; top_p=0.95; max_new_tokens=40 | Samples among filtered candidate tokens. Outputs can vary; useful to understand generation, not a controlled quality comparison. |
| Greedy | do_sample=False; num_beams=1; max_new_tokens=40 | Chooses the highest-probability next token. Simple and inexpensive baseline. |
| Beam search | do_sample=False; num_beams=4; early_stopping=True; max_new_tokens=40 | Maintains several candidate sequences. More search/compute, without a guarantee of better meaning. |

## Actual five-case comparison

- Both strategies produced "Le chat dorme sur le canapé." for "The cat is sleeping on the sofa." In a simple affirmative sentence, "Le chat dort ..." is the expected indicative form.
- Both preserved "2" and "500 mL" in the bottles example. This single case does not prove number/unit reliability.
- For "Tide laundry detergent is on sale.", greedy produced "Le détergent de blanchisserie à tuyauterie est en vente." Beam produced "Le détergent de blanchisserie est en vente." Neither preserved Tide.
- The delivery-fee and battery-recharging examples produced identical outputs across strategies. Context distinguished the two senses of charge in these probes.
- Only 1 of 5 outputs changed between strategies. That count is not an improvement rate.

## Generation limits

max_length caps total generated decoder sequence length; max_new_tokens caps new tokens after the decoder prefix. The notebook checks at most 41 output IDs for 40 new tokens plus T5's initial decoder token. It also checks whether EOS appeared, since reaching a cap can leave a sentence unfinished.

## How to compare the fine-tuned model - proposed

Load both tokenizer and model from outputs/t5-small-en-fr/final, then evaluate the same inputs and decoding settings as the pretrained checkpoint. Use deterministic generation for a controlled first comparison and record quality plus measured timing. Do not describe the current notebook 4 comparison as a fine-tuning comparison.

The saved inference checks passed for batch size, nonempty output, device, evaluation mode, input/output limits, and EOS. These establish operational consistency, not accuracy.

# Retail quality and the HOLD decision

| Case | What is being checked |
|---|---|
| Quantity | Buy 2 bottles of water: preserve the count and object. |
| Unit | A bag with 5 kg of rice: preserve amount and unit. |
| Brand | LEGO set with 120 pieces: preserve LEGO and the count. |
| Return policy | Unopened items within 30 days: preserve condition and deadline. |
| Price | 12.50 euros -> 12,50 euros: preserve value with locale formatting. |
| Negation | Do not use on wet surfaces: preserve the prohibition and surface condition. |

Saved results: all six automatic checks passed; retail corpus BLEU was 72.56; mean warm request latency was 0.114 seconds and maximum was 0.146 seconds. These are recorded local observations, not service guarantees.

## How the checks work

Each case passes automatically only if output is nonempty, numeric tokens match, required term alternatives are found, and input was not truncated. Number checking compares multisets of Decimal values and accepts decimal commas/dots. It does not understand number words, thousands separators, or which number belongs to which attribute. Term presence does not prove the whole sentence is correct.

## Why HOLD was correct

> Human-reviewed cases = 0. human_review values remain None. latency_budget_seconds is None. The saved decision is HOLD because human review is incomplete and no latency target has been set. Six automatic passes do not override these missing requirements.

The notebook can report READY FOR A LARGER PILOT only when all automatic conditions pass, every human review is True, a positive latency target is supplied, and the maximum observed latency fits it. That label still does not authorize production release.

## Timing and cost: understand the denominator

There is one warm-up call, then one measured call per case. Timing includes tokenization, generation, and decoding on the local machine; it excludes model loading, networking, queueing, and a real service stack. Six sequential requests do not estimate p95/p99 latency or concurrent throughput.

```
illustrative cost per 1,000 sequential requests
  = mean_seconds * 1000 / 3600 * hourly_compute_cost
```

hourly_compute_cost_usd remains unset, so no cost is established. The formula assumes continuously billed sequential compute and excludes idle time/service overhead. For production, measure cost per accepted translation as well as throughput and latency.

# Debugging and reproducibility

## The -100 decoding issue

The current metric callback converts -100 in both predictions and labels into pad_token_id before decoding. -100 means ignore this label in the loss; it is not a legal token ID. Trainer may also introduce this padding while combining generated batches. The notebook contains a synthetic regression check with unequal lengths and -100 padding; the saved check returns BLEU 100 for identical decoded text.

> Debugging explanation: "I would inspect the failing array and its values, identify the difference between tokenizer padding and loss-ignore padding, then test a tiny unequal-length case. Sanitizing both predictions and labels makes decoding safe without changing the intended text." The code and saved checks support this explanation; they do not establish who originally diagnosed the issue.

## Layers of verification

| Layer | Evidence and limitation |
|---|---|
| Environment tests | Setup record reports 19 passing tests. Imports, tensors, installed APIs, a tiny random T5, metric, notebook schemas, and actual kernel startup are checked. This is a historical setup result, not a fresh run for this guide. |
| Evaluation notebook | Saved output says all 12 consistency checks passed. These cover IDs, tokenized subsets, decoding padding, table alignment, numeric ranges, flags, reviews, and decision logic. |
| Inference notebook | Saved structural generation checks pass. They do not evaluate French correctness. |
| Scientific validity | Still needs content-level leakage checks, representative data, language alignment, full-reference review, uncertainty, and bilingual judgments. |

## Environment facts to know

The lab records Python 3.12.10, PyTorch 2.14.1+cpu, Transformers 5.19.0, Datasets 5.1.0, and SacreBLEU 2.6.0. A laptop NVIDIA GPU was detected, but the installed PyTorch build is CPU. Hardware presence does not mean CUDA training was enabled.

## Common failure -> first check

- Missing package or wrong behavior -> inspect sys.executable in both notebook and terminal; interpreter and notebook kernel are separate selections.
- Variable missing -> run notebook setup cells in order; state is not shared between notebooks.
- Windows cache/path failure -> use the normal shorter Hugging Face user cache described in README.
- Wrong checkpoint -> print the loaded path/name and load matching tokenizer assets.
- Stale outputs -> restart and run in order when deliberately reproducing; notebook 3 currently retrains if its training cell is executed.

For future reproducibility, save dataset revision and selected IDs, model revision, source commit, dependencies, seed, metric signature, generation configuration, and output text. A seed alone does not guarantee identical results across hardware or library versions.

EXTENSIONS • NOT COMPLETED IN THIS LAB

# The next experiments to propose

## Prioritize trustworthy measurement

- 1. Repair evaluation data: audit swapped-language rows, alignments, exact/near duplicates, and truncation. Split at a suitable document or product-family level before selecting subsets.
- 2. Define a retail evaluation set with bilingual references. Cover product categories, source lengths, language/locale variants, brands, numbers, policy conditions, and hard negatives. Freeze the final test set.
- 3. Re-establish the pretrained baseline on the cleaned validation set with fixed decoding and a saved metric signature. Add severity-based human review.
- 4. Compare the fine-tuned checkpoint under the identical evaluation protocol. Increase data or adjust one training choice at a time, keeping the same comparison set.
- 5. Benchmark quality, repeated warm/cold latency, throughput under load, and cost. Choose from measured tradeoffs, then use the held-out test set once the choice is frozen.

| Hypothesis | Controlled comparison | Decision evidence |
|---|---|---|
| Domain data helps retail fidelity | Same initialization and evaluation; general-domain vs curated retail fine-tuning | Critical errors, terminology fidelity, human adequacy, corpus metrics. |
| Glossary support preserves brands | Same examples/model; baseline vs glossary or protected-span strategy | Brand/SKU preservation plus grammar and overall meaning. |
| More search helps enough to justify cost | Greedy vs four beams on the same checkpoint and examples | Quality change versus latency/throughput. Current five-case test does not settle this. |
| An alternative model is a better fit | Task-oriented seq2seq vs an appropriately selected LLM | Language coverage, fidelity, latency, cost, reproducibility, operational fit. |

## Research habits worth emphasizing

Predefine the evaluation and success criterion; make one interpretable change at a time; keep failure examples; report negative results; and check whether improvements hold across relevant slices. Multiple training seeds assess training variability. Paired evaluation comparisons control for which examples were tested.

Possible extensions include parameter-efficient adaptation, distillation, quantization, and terminology constraints. None is implemented or measured here. Select one only after the baseline reveals the actual quality or resource bottleneck.

If asked "What would you do with another week?": "First fix evaluation integrity, then establish a reviewed retail baseline, then run one controlled adaptation experiment. I would prioritize a trustworthy result over a larger but uninterpretable run."

PROPOSED SYSTEM DESIGN FOR THE ROLE

# From a notebook to a translation service

The job description mentions real-time and batch multilingual experiences. The following is a discussion design, not an architecture deployed by this repository. Begin by asking about supported languages/locales, content types, traffic, latency, quality risk, and human-review capacity.

```
OFFLINE
source content -> validate/deduplicate -> versioned data
  -> train/adapt -> offline evaluation + bilingual review
  -> versioned model/tokenizer/glossary -> staged release

ONLINE
request + locale -> validate -> translation cache
  -> model routing/generation -> fidelity checks
  -> return result OR reviewed fallback / human queue
  -> metrics and sampled error review
```

## Real-time path

Load models once per worker. Set input limits and timeouts, batch compatible requests when latency permits, cache repeated content, and monitor queueing. Include source text, source/target locale, model version, and glossary version in cache identity. A changed product attribute or policy must not reuse a stale translation.

## Batch path

Use a job queue for catalog ingestion, stable content IDs, idempotent writes, checkpoints, and bounded retries. Batch by similar sequence lengths to reduce wasted padding. Persist source version, model version, output, checks, and review status. Reprocess only affected items when content or glossary versions change.

## Quality controls and operations

- Use reviewed terminology resources for brands, units, policy phrases, and locale conventions. Protect structured values carefully and verify their reinsertion.
- Route high-risk or failed checks to an appropriate review/fallback path. Agree behavior with product stakeholders; do not silently return uncertain policy text.
- Measure p50/p95/p99 latency, throughput, timeouts, cost, and error categories by language, locale, and content type.
- Release gradually with rollback support. Evaluate changes offline first; assess user outcomes with an appropriate controlled online design after quality gates pass.

## Translate business needs into requirements

Example: for product descriptions, agree acceptable critical-error rates, brand/attribute preservation, latency, and review burden. Potential downstream measures include task completion, search success, customer support contacts, and returns associated with misleading content. These are proposed metrics; this lab measures no customer or revenue impact.

> Tradeoff to explain: dynamic batching can improve throughput but add waiting time. Beam search can spend more compute without fixing factual or terminology errors. Measure the service workload rather than assuming the laptop timings transfer.

ROLE-SPECIFIC PREPARATION • PROPOSED EXTENSION

# Localization and Haitian Creole

The supplied role explicitly lists English and Haitian Creole fluency. This project demonstrates English-to-French experiments only. French results do not establish Haitian Creole capability, linguistic fluency, or transfer quality.

## Translation versus localization

Translation preserves meaning across languages. Localization also considers the intended audience and regional wording, formatting, conventions, and product terminology. The job description highlights regional vocabulary differences as a reason multilingual experiences need more than a single literal wording.

## How to adapt this workflow responsibly

- Clarify the intended locale, register, audience, content type, and approved terminology with language specialists and stakeholders.
- Select candidate models based on documented language support, then establish English-Haitian Creole baselines. Do not assume the French task prefix or checkpoint will work.
- Collect or curate aligned data and bilingual review sets with appropriate usage rights and consistent language labels. Check spelling variation, code-switching, named entities, and alignment.
- Use bilingual reviewers to evaluate meaning, naturalness, omissions, and critical retail details. Record disagreement and adjudicate difficult cases.
- Test whether automatic metrics and language-ID tools work adequately for this language/domain. Treat their scores as fallible signals.
- Evaluate adaptation on representative content, retain a held-out test set, and inspect whether gains generalize beyond the training distribution.

## Model comparison answer

"I would compare a sequence-to-sequence model with suitable language coverage and an LLM candidate on the same reviewed examples. I would evaluate fidelity, terminology, critical errors, latency, and cost. A larger model may offer useful context handling, but I would measure whether that helps the retail task enough to justify its operational cost."

## If asked about limited parallel data

Proposed options include multilingual transfer, carefully curated domain examples, active selection of uncertain/high-impact examples for human translation, and back-translation. Back-translation uses target-language monolingual text to create synthetic source text; it can expand training material but can also introduce model errors. Keep synthetic data traceable and out of the final human-reviewed test set.

## A credible answer about the current gap

> "This lab is an English-French prototype. Its evaluation and engineering methods transfer, but the language result does not. For Haitian Creole I would establish a fresh baseline with appropriate model coverage and bilingual review. I would describe my own language proficiency separately and accurately."

Prepare a concrete localization example from your own actual language experience if you have one. Do not memorize an unfamiliar translation as a substitute for language understanding.

ACTIVE RECALL • ANSWER BEFORE READING

# Practice the interviewer follow-ups

| Question | A concise answer to practice |
|---|---|
| Why T5-small? | A manageable pretrained encoder-decoder baseline for a local CPU learning experiment. The choice was not established as optimal for Walmart or Haitian Creole. |
| What is your largest data concern? | Content leakage and language alignment. Initial splits share 119 distinct pairs; one displayed evaluation row reverses English/French direction. |
| Why a task prefix? | T5 uses text-to-text task conditioning. The prefix indicates English-to-French translation. It does not create language capability by itself. |
| Why dynamic padding? | It pads only to the longest sequence in a batch, reducing unnecessary padding. With batch size 1 there is little inter-example padding to save. |
| Why are labels -100? | Those positions are ignored by the loss. Replace/filter -100 before decoding; also handle it in Trainer-generated arrays. |
| Why did BLEU fall while loss fell? | Loss scores reference-token probabilities under teacher forcing. BLEU scores decoded sequences. The objectives differ and this validation set is tiny and imperfect. |
| Did fine-tuning improve your model? | It did not demonstrate an improvement in generated translation quality: validation BLEU fell 0.6263 points. |
| Did beam search fix the brand error? | No. The output changed, but both strategies failed to preserve Tide in the saved example. |
| Can you ship because all retail checks pass? | No. There are only six authored cases; zero recorded human reviews and no agreed latency target. The notebook says HOLD. |
| What does 0.114 seconds mean? | Mean of six warm sequential local requests including tokenization/generation/decoding. It is not networked service latency or p95. |
| What would you fix first? | Evaluation integrity: language direction, content-level splits, representative references, and truncation; then compare checkpoints fairly. |
| What did the tests prove? | The environment and key notebook invariants work in recorded checks. They do not prove accuracy, fluency, or production readiness. |

## Two deeper drills

1. Explain the pipeline in two minutes without library names. Then repeat with the exact functions and tensor fields. This checks whether you understand both concepts and code.

2. Use the Tide failure as a five-minute case: observe the error, propose causes, distinguish evidence from hypotheses, design a controlled glossary or data experiment, and define a quality/latency acceptance criterion.

# Study plan and source trail

The project README identifies Friday, October 16 as the interview date. Use this schedule if that date is still correct; the user-supplied job description does not specify an interview date.

| When | Focus and self-test |
|---|---|
| Tuesday, October 13 | Pages 1-6. Draw the pipeline, explain every tensor field, and distinguish ID separation from text deduplication. |
| Wednesday, October 14 | Pages 7-12. Rehearse the loss/BLEU result, language-direction defect, Tide error, and HOLD decision using exact evidence. |
| Thursday, October 15 | Pages 13-17. Practice proposed experiments, real-time/batch design, and Haitian Creole adaptation. Give a mock 90-second project answer. |
| Friday, October 16 | Review page 1 and the question table. Focus on demonstrated work, limitations, and one clear next experiment. |

## Questions you can ask the interviewer

- Which language/locale pairs and retail content types create the most difficult quality problems today?
- How does the team combine automatic metrics, bilingual review, and product outcomes?
- How do real-time and batch workloads differ in latency and quality requirements?
- How are terminology decisions and critical translation errors handled with product and language experts?

## Local evidence used throughout

N1: notebooks/01_dataset_exploration.ipynb, split and quality-check outputs. N2: notebooks/02_tokenization.ipynb, preprocessing/collation and decoded tensors. N3: notebooks/03_model_evaluation.ipynb, trainer settings, before/after table, error/retail tables, checks, and decision. N4: notebooks/04_model_inference.ipynb, loaded model, generation settings, comparison table, and checks.

Supporting artifacts: outputs/t5-small-en-fr/checkpoint-128/trainer_state.json; final checkpoint file inventory; tests/test_environment.py; README.md; requirements.txt; outputs/SETUP_VERIFICATION.md and environment-versions.json. Setup records describe an earlier state; the current notebooks take precedence for implemented experiments. JD: C:/Users/admin/Downloads/ML_Scientist_JD_Translation.pdf.

## Primary references for further study

[R1] Raffel et al., T5: Exploring the Limits of Transfer Learning (JMLR).
[R2] SacreBLEU official repository: metrics, signatures, and evaluation.
[R3] Post, A Call for Clarity in Reporting BLEU Scores.
[R4] COMET official documentation.
[R5] Hugging Face translation tutorial. Follow the installed project APIs when examples differ.

Final recall: 127,085 pairs; 128 train / 32 validation; 1 epoch; CPU; BLEU 16.5116 -> 15.8853; 119 shared pairs in the initial split; 6 automatic retail passes, 0 human reviews; HOLD; notebook 4 uses pretrained weights.